

Relational Schema

Users (UserID: INT [PK], Name: VARCHAR(250), Email: VARCHAR(250))

CommunityReport (ReportID: INT [PK], UserID: INT [FK to Users.UserID], Latitude: FLOAT, Longitude: FLOAT, ReportText: TEXT, CreatedAt: TIMESTAMP)

ObservationList (ListID: INT [PK], UserID: INT [FK to Users.UserID], Latitude: FLOAT, Longitude: FLOAT, ListName: VARCHAR(250), CreatedAt: TIMESTAMP)

LightPollutionObservation (ObservationID: INT [PK], Latitude: FLOAT, Longitude: FLOAT, LimitingMag: FLOAT)

CelestialObject (ObjectID: INT [PK], Name: VARCHAR(250), Magnitude: FLOAT, ObjectCategory: VARCHAR(250), RightAscension: FLOAT, Declination: FLOAT, Constellation: VARCHAR(250))

SavedObject(ListID: INT [PK] [FK to ObservationList.ListID], ObjectID: INT [PK] [FK to CelestialObject.ObjectID], ObservedStatus: VARCHAR(250), AddedAt: TIMESTAMP)

Entities

1. Users
2. LightPollutionObservation
3. ObservationList
4. SavedObject
5. CelestialObject
6. CommunityReport

Users

A user is a Person using the app, and rows are created/updated/deleted when someone makes an account, updates their account, or deletes their account. We're assuming that emails are unique and will enforce this constraint. Users should be entities and not attributes because each user has multiple pieces of associated information. Furthermore, Users need to be associated with both CommunityReports and ObservationLists since Users can create those entities.

Example Row

<u>UserID</u> (PK)	Name	Email
1	'Jake'	'jake@gmail.com'

LightPollutionObservation

A LightPollutionObservation corresponds with a real world measurement of light pollution in a particular area. Each row in this table comes from a large CSV dataset by [Globe At Night](#) (identified in stage 1). Each LightPollutionObservation has several pieces of associated data, justifying its existence as an entity. Although the dataset provides unique ObservationIDs, these aren't unique to a location because a location can have multiple measurements corresponding with different viewing conditions (like cloud cover). In situations when querying LightPollutionObservation returns multiple results for one location, we'll either choose the average LimitingMag or the largest LimitingMag. The option we choose will be decided in future work for Stage 3. Since Latitude and Longitude are not unique in LightPollutionObservation, we chose not to connect this table with others via direct relationships. Not all locations (combinations of latitude and longitude) are represented in the data, so when the app queries for a particular location's light pollution, we'll also have to look for latitudes and longitudes within a threshold radius (e.g. 30 miles). If no row exists in the data within the threshold radius of the user's location, we plan to return a "default" LimitingMag equal to the median LimitingMag of all rows in the table. We also assume that LimitingMag isn't unique because multiple locations can have the same amount of light pollution.

Example Row

<u>ObservationID</u> (PK)	Latitude	Longitude	LimitingMag
25	40.00	-80.00	5.0

ObservationList

An ObservationList is a User's saved list of celestial objects that they want to see. Rows in ObservationList are created/updated/deleted by Users. An ObservationList is its own entity because it acts as a checklist of astrological objects, and it's described by attributes like its name, when it was created, and location (represented by Latitude and Longitude). We want a user to be able to create multiple ObservationLists, and each ObservationList should be able to hold multiple CelestialObjects that the user wants to see. We assume that a User can make zero or more ObservationLists. We assume that UserID, Latitude, Longitude, ListName, and CreatedAt won't be unique as individual attributes. However, the combination of UserID and CreatedAt will be unique because a user can't make more than one list at a time.

Example Row

<u>ListID</u> (PK)	UserID (FK)	Latitude	Longitude	ListName	CreatedAt
10	1	45.10	-80.05	'Wednesday List'	2026-06-19 11:36:00

SavedObject

A SavedObject is an item representing a saved CelestialObject in a User's ObservationList. Therefore, rows in SavedObject will be created/updated/deleted by Users. Even though a SavedObject isn't exactly a "real world" entity on its own, it needs to exist to create the many-to-many relationship that exists between CelestialObjects and ObservationLists. Furthermore, SavedObjects have their own attributes (ObservedStatus and AddedAt) and are unique to a user and a particular list. Therefore, SavedObjects should be modeled as an entity, not an attribute. We assume that Users can add zero or more SavedObjects to their ObservationLists and that each SavedObject will represent only one CelestialObject. The ObservedStatus attribute can be null (if the user hasn't saved any text), but AddedAt can't be (when a SavedObject is added, the timestamp for AddedAt is generated for it). We assume that AddedAt isn't unique because a user should be able to add multiple celestial objects to their ObservationList simultaneously. ObservedStatus won't be unique because a user can mark multiple items with 'Seen' or 'Not seen', for example.

Example Row

ListID and ObjectID (PK)			
ListID (FK)	ObjectID (FK)	ObservedStatus	AddedAt
10	1	'I saw this'	2026-06-21 11:37:00

CelestialObject

CelestialObjects are real world objects in space like planets in our solar system, stars, and galaxies. Each row in the CelestialObject table will come from the [HYG Database](#) or an [NGC/Messier catalog](#) (two of our data sources from stage 1). Since each CelestialObject is unique and described by numerous attributes, they need to be entities. Some CelestialObjects in our datasets don't have a Name or Constellation, so we assume that those attributes can be null. That's because not every celestial object has a "common name", celestial objects that aren't stars don't belong to constellations, and not all stars belong to constellations. When a CelestialObject's Name is null, it will be referred to by its ObjectID, which is unique. RightAscension and Declination are not null, and the combination of these two attributes is unique because no two celestial objects can occupy the same spatial coordinates.

Example Row

ObjectID (PK)	Name	Magnitude	ObjectCategory	RightAscension	Declination	Constellation
1	'Sirius'	2.5	'Star'	5.572173	-19.49884	'Psc'

CommunityReport

CommunityReports are notes posted by Users about light pollution or other conditions that affect stargazing in a particular area. Rows in CommunityReport will be created/updated/deleted by Users. CommunityReports will be uniquely identifiable, and are described by attributes like Latitude, Longitude, and CreatedAt (creation time), so they should be entities. Similar to ObservationLists, we assume that UserID, Latitude, Longitude, ReportText, and CreatedAt won't be unique as individual attributes. However, the combination of UserID and CreatedAt will be unique because a user won't be allowed to submit more than one CommunityReport per second.

Example Row

ReportID (PK)	UserID (FK)	Latitude	Longitude	ReportText	CreatedAt
1	1	40.70	-31.00	'A bright light has been added to this parking lot, creating additional light pollution.'	2026-06-21 11:38:00

Relationships

User to CommunityReport

Cardinality: one to many

Description: The User to CommunityReport relationship is one-to-many because a user can make many CommunityReports, and we have decided that CommunityReports can't be authored by more than one user.

User to ObservationList

Cardinality: one to many

Description: The User to ObservationList is also one-to-many because a user can make many ObservationLists for themselves, and we're choosing not to allow multiple Users to edit the same ObservationList.

ObservationList to SavedObject

Cardinality: one to many

Description: The ObservationList to SavedObject relationship is one-to-many because an ObservationList should be able to hold multiple items, which are represented by SavedObjects. Also, one SavedObject can't belong to multiple ObservationLists because it will have different attributes (ObservedStatus and AddedAt) depending on the list it's in, even if those lists are created by the same user.

ObservationList to CelestialObject

Cardinality: many to many

Description: The ObservationList to CelestialObject relationship is many-to-many via the SavedObject table because an ObservationList can indirectly reference multiple CelestialObjects, and one CelestialObject can be on many different ObservationLists.

CelestialObject to SavedObject

Cardinality: one to many

Description: The CelestialObject to SavedObject relationship is one to many because one CelestialObject can be referenced by many SavedObjects. This represents a CelestialObject being on many ObservationLists. SavedObjects shouldn't reference more than one CelestialObject at a time because the user needs to track the ObservedStatus of individual CelestialObjects.

Normalization Proof

Result

Since all constituent relation tables satisfy the requirements of Boyce-Codd Normal Form (BCNF), the overall schema is in BCNF, ensuring a design free of redundancy.

Users(UserID, Name, Email)

Candidate Key (CK): Email–UNIQUE

(See assumption 1)

Functional Dependencies (FDs):

- UserID → Name, Email ✓ BCNF since determinant is a superkey
- Email → UserID, Name ✓ BCNF since determinant is a superkey

Normalization: BCNF

LightPollutionObservation(ObservationID, Latitude, Longitude, LimitingMag)

CK: primary key only

(2)

FDs:

- ObservationID $\rightarrow$ Latitude, Longitude, LimitingMag
✓ BCNF since determinant is a superkey

Normalization: BCNF

No other non-trivial dependencies exist, as Latitude, Longitude, and LimitingMag are non-unique and have no functional dependencies among themselves.

ObservationList(ListID, UserID, Latitude, Longitude, ListName, CreatedAt)

CK: UserID + CreatedAt–UNIQUE

(3)

FDs:

- ListID $\rightarrow$ UserID, Latitude, Longitude, ListName, CreatedAt
✓ BCNF since determinant is a superkey
- UserID + CreatedAt $\rightarrow$ ListID, ListName, Latitude, Longitude
✓ BCNF since determinant is a superkey

Normalization: BCNF

UserID is a foreign key that implies nothing beyond what is already captured by the FDs above. ListName, Latitude, Longitude, and CreatedAt are non-unique and have no functional dependencies between them.

SavedObject(ObjectID, ListID, ObservedStatus, AddedAt)

CK: primary key only

FDs:

- ListID + ObjectID $\rightarrow$ ObservedStatus, AddedAt
✓ BCNF since determinant is a superkey

Normalization: BCNF

SavedObject is a relational table between ObservationList and CelestialObject with metadata attributes ObservedStatus and AddedAt which are non-unique and functionally independent.

CelestialObject(ObjectID, Name, Magnitude, ObjectCategory, RightAscension, Declination, Constellation)

CK: RightAscension + Declination–UNIQUE

(4) (5)

FDs:

- ObjectID $\rightarrow$ Name, Magnitude, ObjectCategory, RightAscension, Declination, Constellation
✓ BCNF since determinant is a superkey
- RightAscension + Declination $\rightarrow$ ObjectID, Name, Magnitude, ObjectCategory, Constellation
✓ BCNF since determinant is a superkey

Normalization: BCNF

Name, Magnitude, ObjectCategory, and Constellation are all non-unique and functionally independent. Also, Name could be NULL, and while Constellation + ObjectCategory + Magnitude might identify a unique star, not all stars are part of a Constellation.

CommunityReport(ReportID, UserID, Latitude, Longitude, ReportText, CreatedAt)

CK: UserID + CreatedAt–UNIQUE

(6)

FDs:

- ReportID $\rightarrow$ UserID, Latitude, Longitude, ReportText, CreatedAt
✓ BCNF since determinant is a superkey
- UserID + CreatedAt $\rightarrow$ ReportID, Latitude, Longitude, ReportText
✓ BCNF since determinant is a superkey

Normalization: BCNF

Latitude, Longitude, and ReportText would all be repeatable at the user's discretion and thus non-identifying.

Assumptions

- (1) Emails must be unique.

- (2) Application will either group Lat + Lon for aggregate LimitMag or return the closest coordinates, LIMIT 1.
- (3) Users can create multiple lists, even with the same name and coordinates, but not two lists at the same time.
- (4) No two celestial objects occupy the same spatial coordinates.
- (5) Name attribute for celestial objects not guaranteed to be unique or fully populated.
- (6) Users can create multiple community reports for the same location or with the same text, but not two reports at the same time.